VIRTUAL CODE

ULTIMATE NEXT JS COURSE WITH PROJECT

Prerequisites for this video

VIRTUAL CODE

- HTML & CSS
- Java Script
- React Js
- Mongo DB

VIRTUAL CODE

ULTIMATE NEXT JS COURSE

LEVEL 1

INTRODUCTION TO NEXT JS

INTRODUCTION TO NEXT JS

WHAT IS NEXT JS ?

- **Next.js is a React-based open-source web framework that enables developers to build fast, scalable, and SEO-friendly web applications with features like server-side rendering (SSR), static site generation (SSG), API routes, and built-in optimization for performance and routing.**

- ✨ In short: *It's React, but with superpowers like SSR, SSG, routing, and full-stack capabilities.*

```
npx create-next-app@latest
```

VIRTUAL CODE

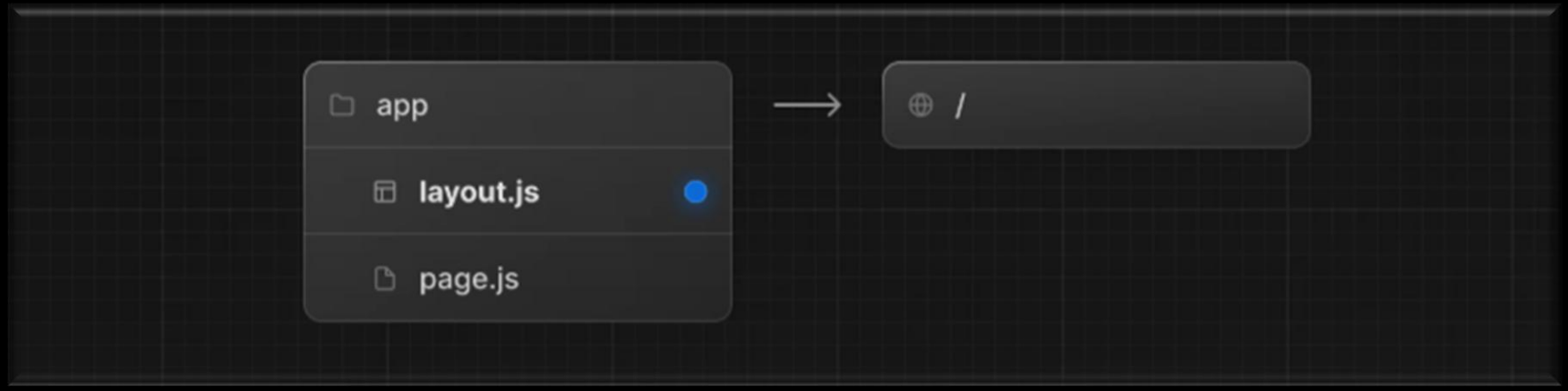
ULTIMATE NEXT JS COURSE

LEVEL 2

App Router

Introduction to App Router -

- Next.js uses file-system routing, which means the routes in your application are determined by how you structure your files.



INTRODUCTION TO NEXT JS

VIRTUAL CODE

App Router

VIRTUAL CODE

Nested Routes-

`folder`	Route segment
`folder/folder`	Nested route segment

App Router

VIRTUAL CODE

Dynamic Routes-

`[folder]`	Dynamic route segment
`[...folder]`	Catch-all route segment
`[[...folder]]`	Optional catch-all route segment

Dynamic Route Segment-

- A Dynamic Segment can be created by wrapping a folder's name in square brackets: [folderName]. For example, a blog could include the following
route app/blog/[slug]/page.js where [slug] is the Dynamic Segment for blog posts.

- Since the params prop is a promise. You must
use async/await or React's use function to access the values.

app/blog/[slug]/page.js JavaScript

```
export default async function Page({ params }) {
  const { slug } = await params
  return <div>My Post: {slug}</div>
}
```

Catch-all Segments

Dynamic Segments can be extended to **catch-all** subsequent segments by adding an ellipsis inside the brackets [...folderName].
For example, app/shop/[...slug]/page.js will match /shop/clothes, but also /shop/clothes/tops, /shop/clothes/tops/t-shirts, and so on.

Route	Example URL	`params`
`app/shop/[...slug]/page.js`	`/shop/a`	`{ slug: ['a'] }`
`app/shop/[...slug]/page.js`	`/shop/a/b`	`{ slug: ['a', 'b'] }`
`app/shop/[...slug]/page.js`	`/shop/a/b/c`	`{ slug: ['a', 'b', 'c'] }`

Optional Catch-all Segments

Catch-all Segments can be made **optional** by including the parameter in double square brackets: [[...folderName]].
For example, app/shop/[[...slug]]/page.js will **also** match /shop, in addition
to /shop/clothes, /shop/clothes/tops, /shop/clothes/tops/t-shirts.
The difference between **catch-all** and **optional catch-all** segments is that with optional, the route without the parameter is also matched (/shop in the example above).

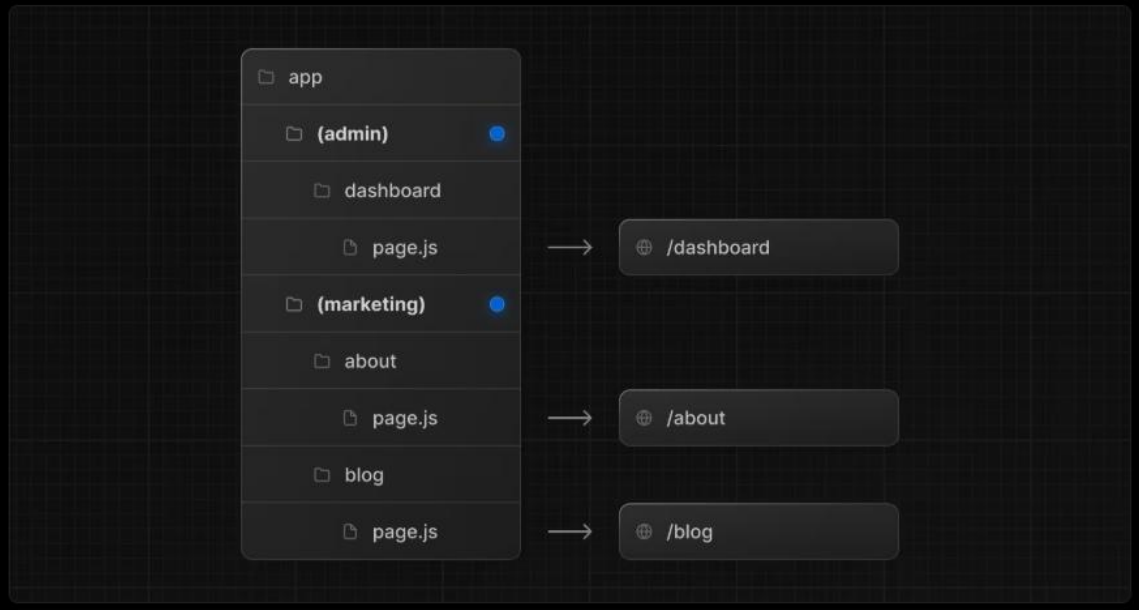
Optional Catch-all Segments

Route	Example URL	`params`
`app/shop/[[...slug]]/page.js`	`/shop`	`{ slug: undefined }`
`app/shop/[[...slug]]/page.js`	`/shop/a`	`{ slug: ['a'] }`
`app/shop/[[...slug]]/page.js`	`/shop/a/b`	`{ slug: ['a', 'b'] }`
`app/shop/[[...slug]]/page.js`	`/shop/a/b/c`	`{ slug: ['a', 'b', 'c'] }`

Routes Group -

A route group can be created by wrapping a folder's name in parenthesis: `(folderName)`.

This convention indicates the folder is for organizational purposes and should **not be included** in the route's URL path.



Parallel Routes -

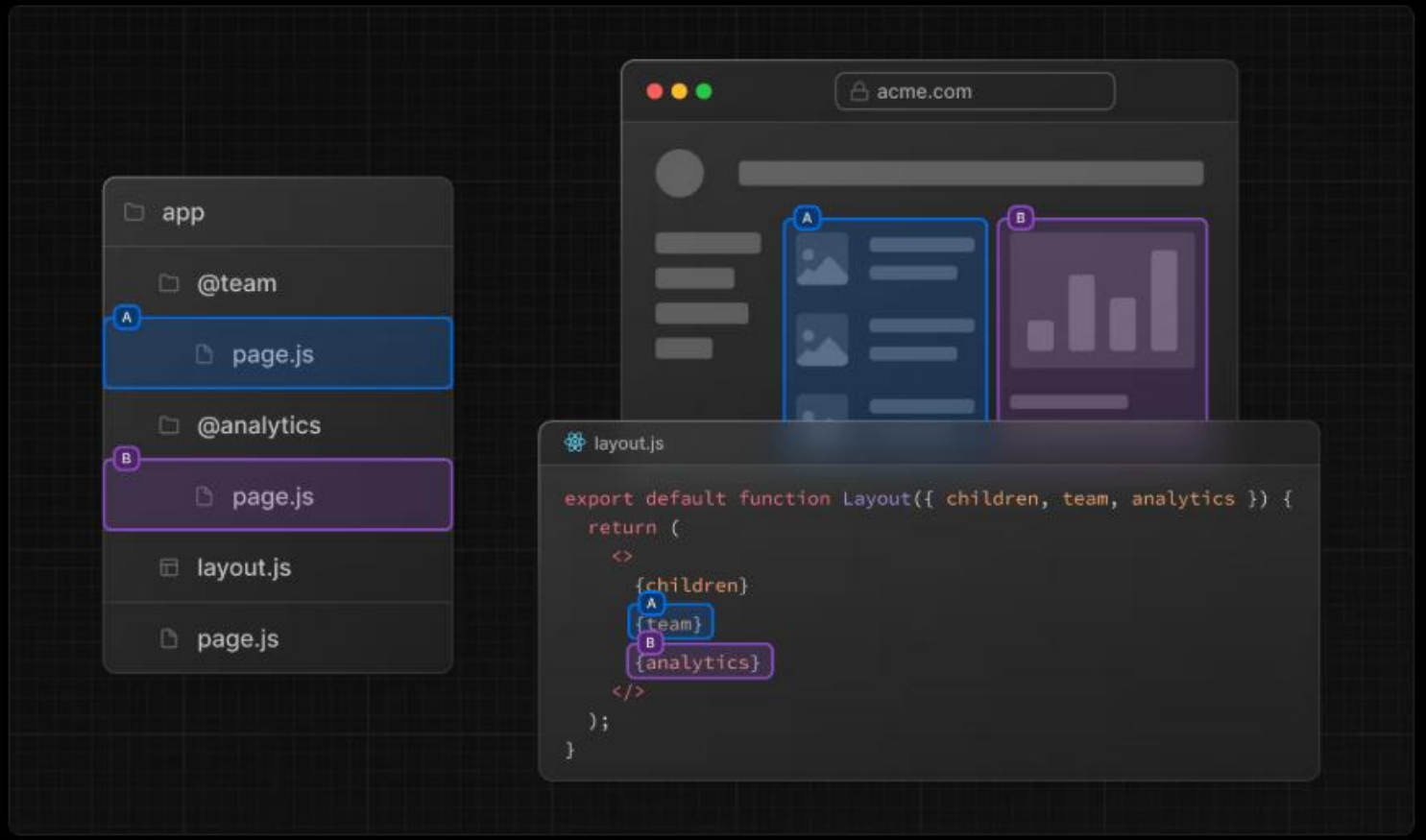
Parallel Routes allows you to simultaneously or conditionally render one or more pages within the same layout. They are useful for highly dynamic sections of an app, such as dashboards and feeds on social sites.

For example, considering a dashboard, you can use parallel routes to simultaneously render the `team` and `analytics` pages:

App Router

VIRTUAL CODE

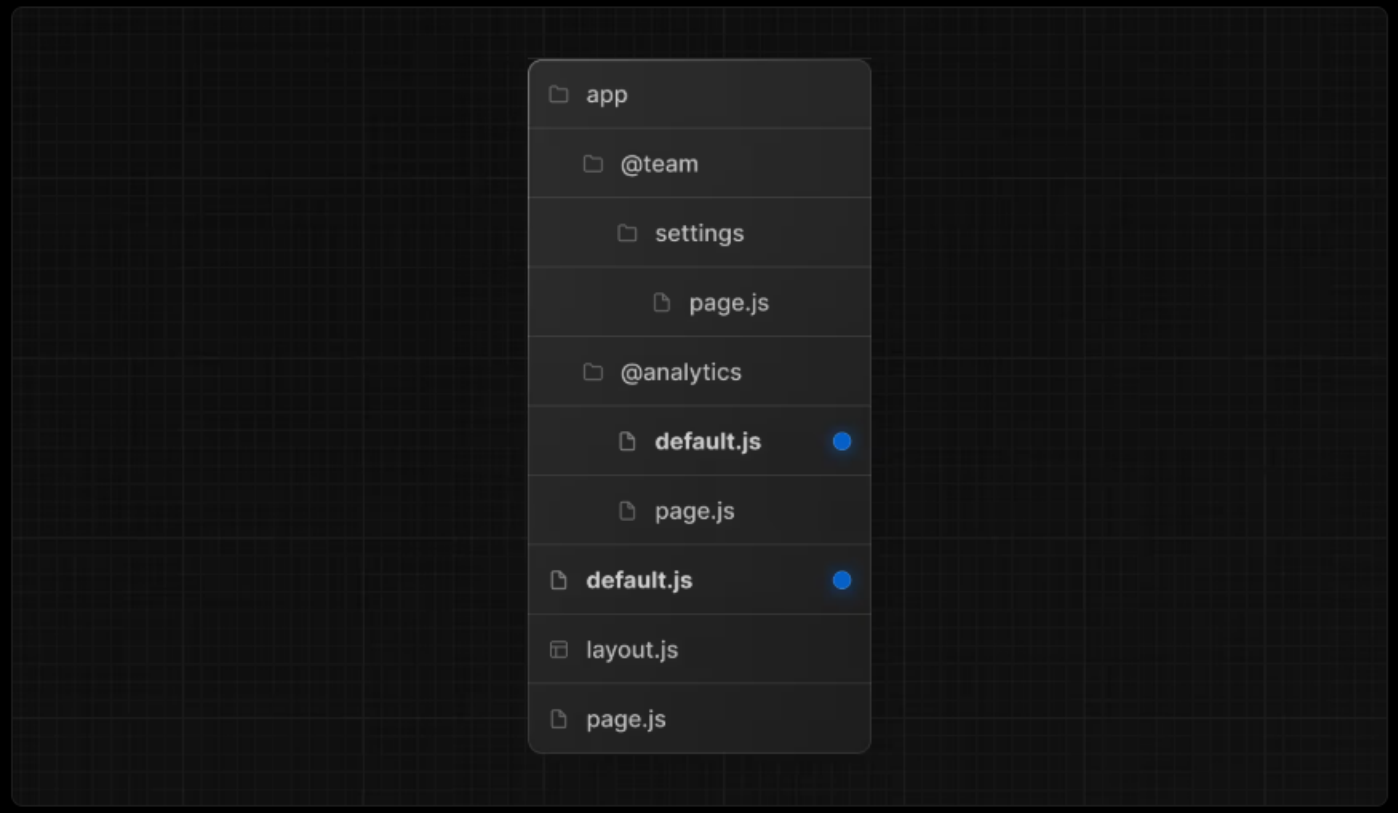
Parallel Routes -



`default.js`

You can define a `default.js` file to render as a fallback for unmatched slots during the initial load or full-page reload.

Consider the following folder structure. The `@team` slot has a `/settings` page, but `@analytics` does not.



App Router

Routing Files

`layout`	`.js` `.jsx` `.tsx`	Layout
`page`	`.js` `.jsx` `.tsx`	Page
`loading`	`.js` `.jsx` `.tsx`	Loading UI
`not-found`	`.js` `.jsx` `.tsx`	Not found UI
`error`	`.js` `.jsx` `.tsx`	Error UI
`global-error`	`.js` `.jsx` `.tsx`	Global error UI
`route`	`.js` `.ts`	API endpoint
`template`	`.js` `.jsx` `.tsx`	Re-rendered layout
`default`	`.js` `.jsx` `.tsx`	Parallel route fallback page

App Router

Note -

In next js , every page is server component so if you want to use hooks (useState…. etc) , you have to write ‘use client’ at the top of the page.

VIRTUAL CODE

ULTIMATE NEXT JS COURSE

LEVEL 3

Navigation , Image & Font in Next JS

Navigation , Image & Font in Next JS

Navigation in Next JS -

There are two methods of Navigation in Next JS

1. **Declarative navigation** → `<Link>` component ke through
2. **Programmatic navigation** → `useRouter()` hook ke through

Navigation , Image & Font in Next JS

VIRTUAL CODE

🔹 1. Declarative Navigation (using `<Link>`)

Ye simple aur recommended method hai — directly UI me link likh dete ho:

jsx — Copy code

```jsx
import Link from 'next/link'

export default function Home() {
  return (
    <div>
      <h1>Home Page</h1>
      <Link href="/about">Go to About Page</Link>
    </div>
  )
}
```

✅ **Benefit:** Fast client-side routing, automatic prefetching, SEO-friendly.

🔹 2. Programmatic Navigation (using `useRouter`)

Kabhi-kabhi hume navigation **automatically** karni hoti hai — jaise:

- Form submit hone ke baad next page par redirect karna,
- Button click se navigate karna (without Link).

Tab hum `useRouter()` hook use karte hain.

Navigation , Image & Font in Next JS

VIRTUAL CODE

jsx

```jsx
'use client'
import { useRouter } from 'next/navigation'

export default function Home() {
  const router = useRouter()

  const goToAbout = () => {
    router.push('/about')
  }

  return (
    <div>
      <h1>Home Page</h1>
      <button onClick={goToAbout}>Go to About</button>
    </div>
  )
}
```

Image Optimization -

Automatically resizes, compresses, and serves images in modern formats (like WebP/AVIF) for faster loading and better performance.

```
import Image from 'next/image'

export default function Home() {
  return (
    <Image
      src="/images/sample.jpg"
      alt="Sample Image"
      width={600}
      height={400}
      priority
    />
  )
}
```

Font Optimization -

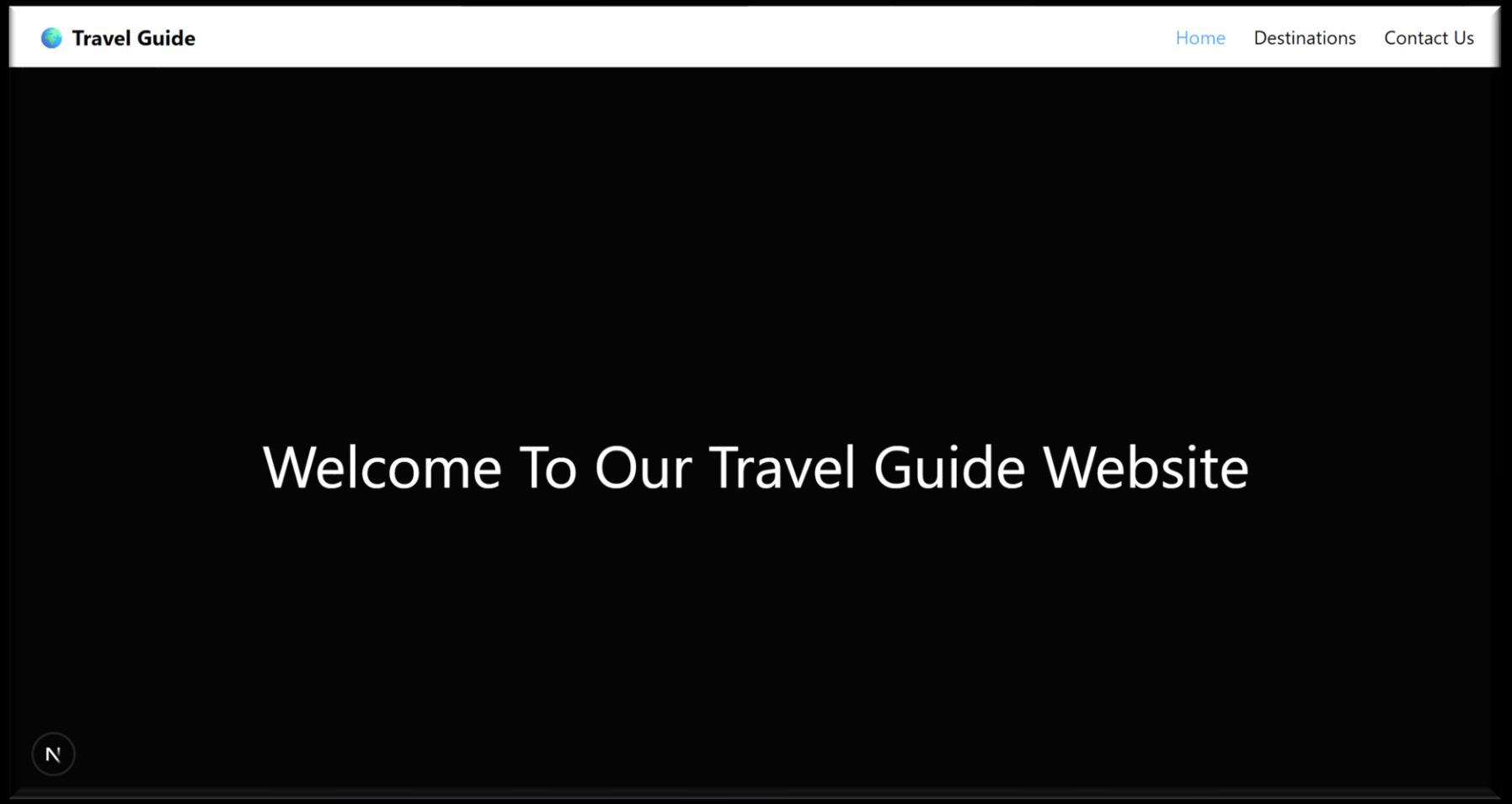
Reduces font file size by including only needed characters (subsets) and preloading them for faster page rendering and minimal layout shifts.

```
import { Roboto } from 'next/font/google'

const roboto = Roboto({ subsets: ['latin'], weight: ['400', '700'], display: 'swap' })

export default function Home() {
  return <h1 className={roboto.className}>Hello, Next.js Fonts!</h1>
}
```

Practice Project 1

VIRTUAL CODE

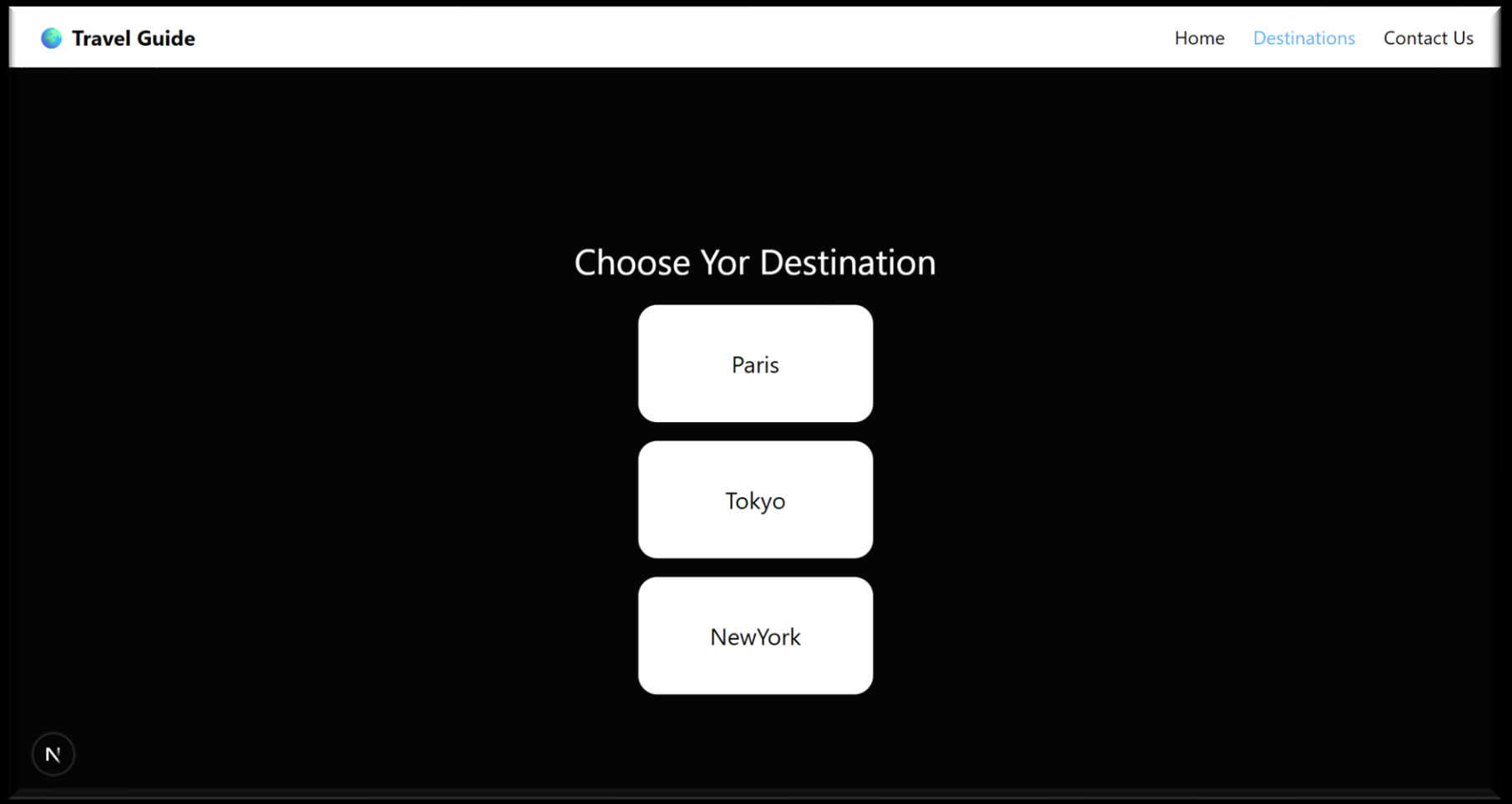
Build a simple Travel Guide website using next JS -



Practice Project 1

VIRTUAL CODE

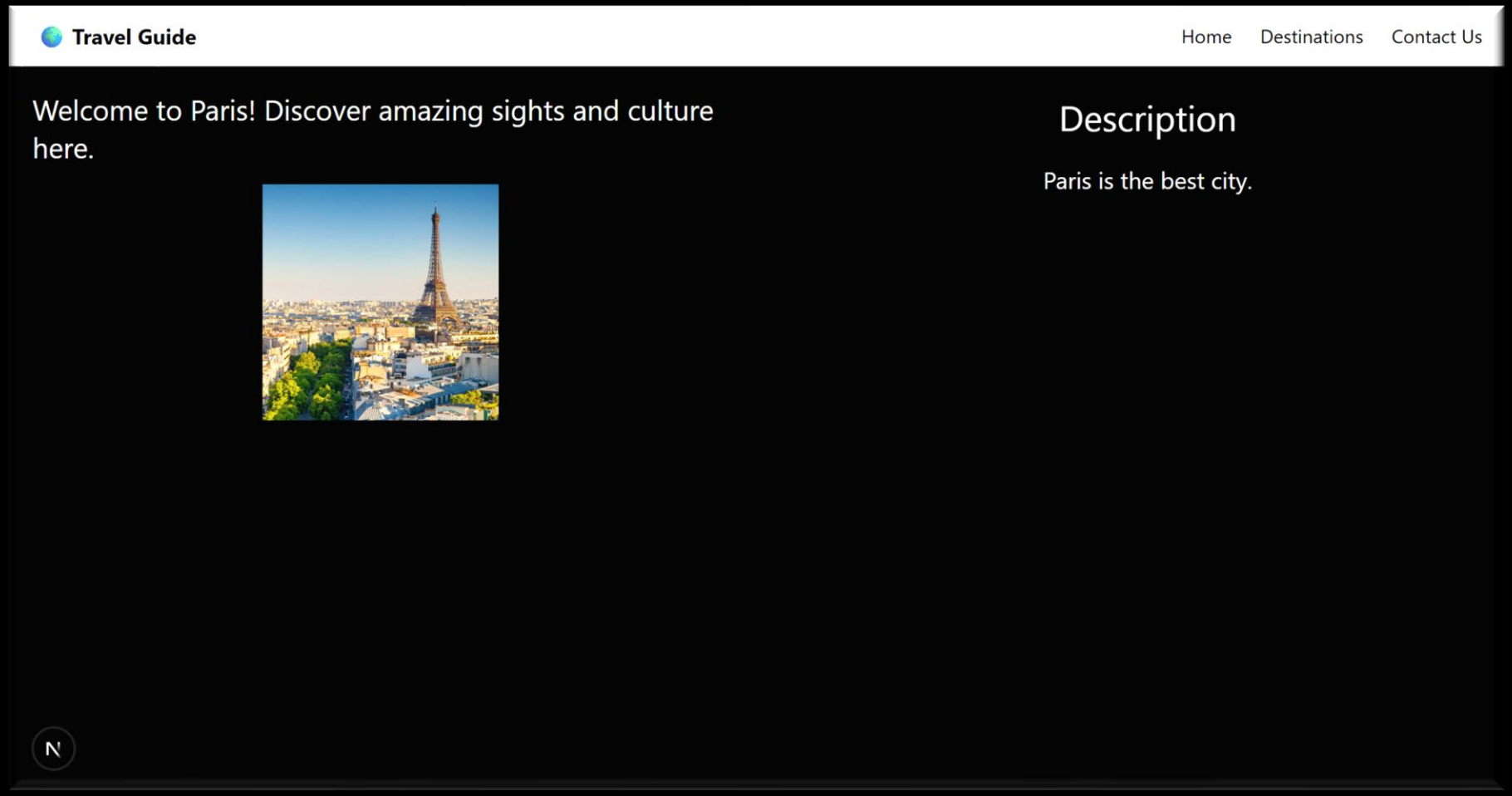
Build a simple Travel Guide website using next JS -



Practice Project 1

VIRTUAL CODE

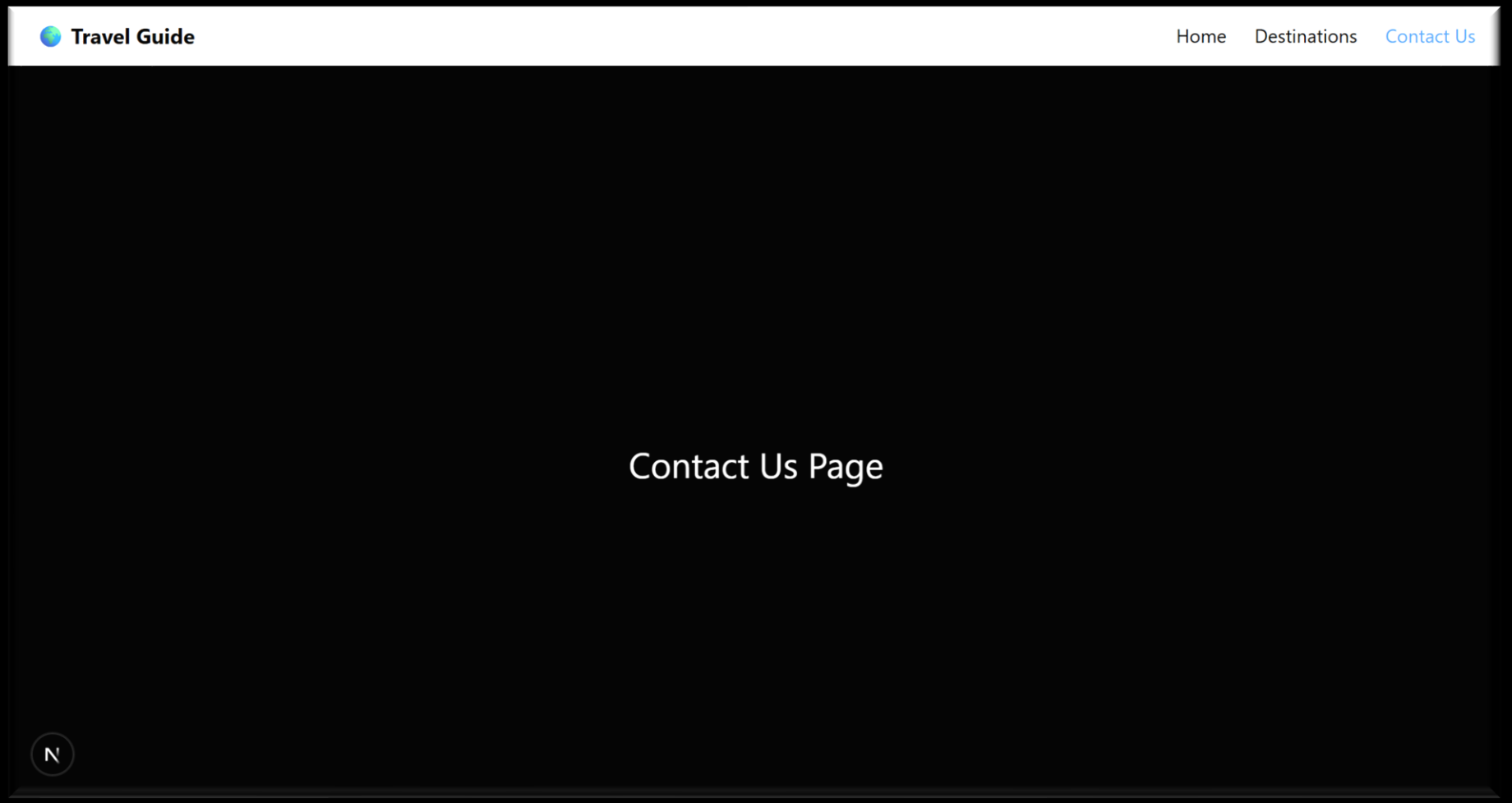
Build a simple Travel Guide website using next JS -



Practice Project 1

VIRTUAL CODE

Build a simple Travel Guide website using next JS -



VIRTUAL CODE

ULTIMATE NEXT JS COURSE

LEVEL 4

Typescript Tutorial

Typescript Tutorial

What is TypeScript ?

- TypeScript is a strongly typed, object-oriented, compiled programming language developed by Microsoft.
- It is a superset of JavaScript, meaning that all valid JavaScript code is also valid TypeScript code.
- The primary addition TypeScript brings to JavaScript is static typing, allowing developers to define the types of variables, function parameters, and return values.

Typescript Tutorial

Basic Types -

Number	`let age: number = 25;`
String	`let name: string = "Aman";`
Boolean	`let isOnline: boolean = true;`
Array	`let scores: number[] = [10, 20, 30];`
Tuple	`let user: [string, number] = ["Aman", 23];`
Enum	`enum Role { Admin, User, Guest }`
Any	`let something: any = "can be anything";`
Void	`function log(): void { console.log("Hi"); }`
Null / Undefined	explicit: `let x: null = null;`

Typescript Tutorial

Arrays & Tuples -

💡 Arrays

ts

```ts
let scores: number[] = [10, 20, 30];
let names: Array<string> = ["Aman", "Ravi"];
```

💡 Tuples

Fixed-length, ordered with specific types.

ts

```ts
let user: [string, number] = ["Aman", 22];
```

Typescript Tutorial

VIRTUAL CODE

Functions -

💡 Basic Functions

ts

```ts
function add(a: number, b: number): number {
  return a + b;
}
```

💡 Optional & Default Parameters

ts

```ts
function greet(name: string, prefix: string = "Hi"): string {
  return `${prefix}, ${name}`;
}
```

Typescript Tutorial

Type Declaration & Type Inference -

Explicit Declaration

ts

```ts
let count: number = 5;
```

Type Inference

ts

```ts
let age = 20;          // inferred number
let city = "Delhi";    // inferred string
function double(x: number) {
  return x * 2;        // inferred number
}
```

Typescript Tutorial

Type Aliases -

Reusable type definitions:

ts

```ts
type ID = number | string;
type Status = "success" | "error" | "loading";

let userId: ID = 42;
let state: Status = "success";
```

Also for function signatures:

ts

```ts
type MathFn = (a: number, b: number) => number;
const divide: MathFn = (a, b) => a / b;
```

Typescript Tutorial

Interfaces -

ts

```ts
interface User {
  id: number;
  name: string;
  email?: string;     // optional
  readonly role: string;
}

const u: User = { id: 1, name: "Aman", role: "Admin" };
```

Extend:

ts

```ts
interface Employee extends User {
  salary: number;
}
```

Typescript Tutorial

Difference b/w Type & Interface -

Feature	type	interface
Combine	`&`	`extends`
Merge	❌	✅
Union	✅	❌
Use-case	primitives, unions, functions	object shapes

ts

```ts
type A = { a: number };
type B = { b: string };
type AB = A & B;

interface X { x: number; }
interface Y extends X { y: string; }
```

Union & Intersection -

```ts

// Union
type Response = "ok" | "fail" | "loading";
let res: Response = "ok";

// Intersection
type Person = { name: string };
type Contact = { phone: string };
type Full = Person & Contact;
const user: Full = { name: "Aman", phone: "123" };
```

Typescript Tutorial

Generics -

Reusable, type-safe structures.

ts

```ts
function identity<T>(value: T): T {
  return value;
}
const n = identity(10);
const s = identity("hello");

// Generic Interface
interface Box<T> { value: T; }
const stringBox: Box<string> = { value: "Hi" };
```

Typescript Tutorial

Global Declaration -

src/types.d.ts

```ts
declare global {
  interface Window { appVersion: string; }

  type ApiResponse<T> = {
    data: T;
    error?: string;
  };
}
export {};
```

Typescript Tutorial

React with TS -

Props

```tsx
type ButtonProps = {
  label: string;
  onClick: () => void;
};
const Button: React.FC<ButtonProps> = ({ label, onClick }) =>
  <button onClick={onClick}>{label}</button>;
```

useState / useRef

```tsx
const [count, setCount] = useState<number>(0);
const inputRef = useRef<HTMLInputElement>(null);
```

Typescript Tutorial

VIRTUAL CODE

Type	Description	Common Use
`ReactNode`	Anything React can render	Children props
`ReactElement`	A single React element instance	One element prop
`JSX.Element`	Return type of component	Function component return
`ReactChild`	string, number, ReactElement	Strict single child
`ReactPortal`	For portals	Portal rendering
`ReactFragment`	For `<>...</>`	Grouping
`ReactComponentType<P>`	Any component with props P	Dynamic rendering
`React.FC<P>`	Function Component type	Components with children

Typescript Tutorial

VIRTUAL CODE

✅ Best Practice Summary

1. children → use `React.ReactNode`
2. element prop → use `React.ReactElement`
3. component return type → use `JSX.Element`
4. dynamic component as prop → use `React.ComponentType<P>`
5. multiple possible types → use `ReactNode | null`

VIRTUAL CODE

ULTIMATE NEXT JS COURSE

LEVEL 5

API Routes In NEXT JS

API Routes in Next JS

VIRTUAL CODE

🧠 What is an API Route?

- An API Route is a **backend function** inside your Next.js app.
- It can **receive requests** and **send responses** (like a mini server).
- You don't need a separate Node/Express server.

bash

```
/app/api/
```

- Each folder can have a `route.js` (or `route.ts`) file.
- Each `route.js` = **one API endpoint**.

Example:

swift

```
/app/api/hello/route.js
```

URL → `/api/hello`

API Routes in Next JS

VIRTUAL CODE

1 Simple GET API → `/api/hello`

ts

```ts
// app/api/hello/route.ts
import { NextResponse } from "next/server";

export async function GET() {
  return NextResponse.json({ message: "Hello from TypeScript API!" });
}
```

2 POST API → `/api/submit`

ts

```ts
// app/api/submit/route.ts
import { NextRequest, NextResponse } from "next/server";

export async function POST(request: NextRequest) {
  const body = await request.json();
  return NextResponse.json({ message: "Data received!", data: body });
}
```

API Routes in Next JS

VIRTUAL CODE

1 Dynamic Route with `NextRequest` → `/api/users/[id]`

ts

```ts
// app/api/users/[id]/route.ts
import { NextRequest, NextResponse } from "next/server";

interface Params {
  params: { id: string };
}

export async function GET(request: NextRequest, { params }: Params) {
  const { id } = params;

  // You can also access request info
  const userAgent = request.headers.get("user-agent");

  return NextResponse.json({ userId: id, userAgent });
}
```

✅ URL: `/api/users/123`

Response:

json

```json
{ "userId": "123", "userAgent": "..." }
```

API Routes in Next JS

2 Query Parameter Route with `NextRequest` → `/api/search?q=nextjs`

ts

```ts
// app/api/search/route.ts
import { NextRequest, NextResponse } from "next/server";

export async function GET(request: NextRequest) {
  // Access query parameter ?q=nextjs
  const query = request.nextUrl.searchParams.get("q");

  // Access headers if needed
  const userAgent = request.headers.get("user-agent");

  return NextResponse.json({ query, userAgent });
}
```

✅ URL: `/api/search?q=nextjs`

Response:

json

```json
{ "query": "nextjs", "userAgent": "..." }
```

VIRTUAL CODE

ULTIMATE NEXT JS COURSE

LEVEL 6

Data Fetching in NEXT JS

Data Fetching in NEXT JS

VIRTUAL CODE

Data Fetching ka matlab hai:

> "Server ya API se data laana taaki page render ho sake."

Next.js me data fetching ke 3 major tareeke hote hain (App Router ke context me):

Type	Full Form	Kab Data Fetch Hota Hai	Use Case
SSR	Server Side Rendering	Har request pe	Dynamic pages (dashboard, user profile)
SSG	Static Site Generation	Build time pe	Blogs, landing pages
ISR	Incremental Static Regeneration	Build time + background me re-generate	News, e-commerce, changing content

Data Fetching in NEXT JS

VIRTUAL CODE

🔹 Simple Analogy

Type	Example Analogy
SSR	Har baar taja khana banana 🍳
SSG	Ek baar bana ke store karna 🍱
ISR	Store kiya hua khana, lekin har kuch der baad naya bana dena 🔁

Data Fetching in NEXT JS

VIRTUAL CODE

🧠 Next.js 15 (App Router) me ye kaise karte hain?

Ab hum `getServerSideProps` ya `getStaticProps` nahi likhte.

Uski jagah **fetch()** ke options se control karte hain:

Type	fetch Option
SSR	`{ cache: 'no-store' }`
SSG	`{ cache: 'force-cache' }` (default)
ISR	`{ next: { revalidate: X } }`

Data Fetching in NEXT JS

VIRTUAL CODE

(A) SSR – Server Side Rendering

Har request pe fresh data laata hai

📁 `app/ssr/page.js`

jsx Copy code

```jsx
export default async function SSRPage() {
  const res = await fetch('https://jsonplaceholder.typicode.com/posts/1', {
    cache: 'no-store', // disables caching
  });
  const data = await res.json();

  return (
    <div>
      <h1>Server Side Rendering (SSR)</h1>
      <p>{data.title}</p>
      <p>Rendered at: {new Date().toLocaleTimeString()}</p>
    </div>
  );
}
```

📝 **Explanation:**

- `cache: 'no-store'` → har request pe API call hoti hai.
- Page refresh karne pe **time badalta dikhega** → proof that it's SSR.

Data Fetching in NEXT JS

VIRTUAL CODE

🟣 (B) SSG – Static Site Generation

Build time pe data fetch hota hai (ek hi baar)

📁 `app/ssg/page.js`

jsx — Copy code

```jsx
export default async function SSGPage() {
  const res = await fetch('https://jsonplaceholder.typicode.com/posts/2', {
    cache: 'force-cache', // default behavior
  });
  const data = await res.json();

  return (
    <div>
      <h1>Static Site Generation (SSG)</h1>
      <p>{data.title}</p>
      <p>Built at: {new Date().toLocaleTimeString()}</p>
    </div>
  );
}
```

📄 **Explanation:**

- Ye page **build ke time** render hota hai.
- Refresh karne pe **same time** rahega (static content).

Data Fetching in NEXT JS

VIRTUAL CODE

🔴 (C) ISR – Incremental Static Regeneration

Static page banta hai, lekin background me auto refresh hota hai

📁 `app/isr/page.js`

jsx — Copy code

```jsx
export default async function ISRPage() {
  const res = await fetch('https://jsonplaceholder.typicode.com/posts/3', {
    next: { revalidate: 10 }, // 10 seconds ke baad background me update
  });
  const data = await res.json();

  return (
    <div>
      <h1>Incremental Static Regeneration (ISR)</h1>
      <p>{data.title}</p>
      <p>Generated at: {new Date().toLocaleTimeString()}</p>
    </div>
  );
}
```

📝 **Explanation:**

- Ye page static hai, lekin **10 seconds** ke baad Next.js background me naya version bana lega.
- Refresh karte raho — 10 sec baad "Rendered at" time change ho jaayega (auto regeneration).

Data Fetching in NEXT JS

VIRTUAL CODE

🧭 3. Summary Table

Concept	Data Fetch Time	fetch Option	Updates?	Example Use
SSR	On every request	`{ cache: 'no-store' }`	Every reload	Dashboard, profile
SSG	At build time	`{ cache: 'force-cache' }`	Never	Blogs, docs
ISR	Build + background regenerate	`{ next: { revalidate: x } }`	Every X seconds	Product pages, news

Practice Project 2

VIRTUAL CODE

Build a Full Stack Next JS App in which –

- Use MongoDB Atlas 
- Authentication Using Next Auth 
- Image Upload Using Cloudinary